WHITE PAPER · APRIL TO JUNE 2025

Advances in Quantization for Large Language Models and Vision-Language Models (2023 to 2025)

Aditya Saxena
In collaboration with UC Irvine Computer Science faculty

ABSTRACT

Large language models and vision-language models are too large to serve cheaply, or to run on consumer and edge hardware, at full precision. This white paper reviews the post-training quantization methods published between 2023 and 2025 that made 4-bit weights, and 8-bit or even 4-bit activations, practical without full retraining: GPTQ, AWQ, OWQ, SpQR, SqueezeLLM, SmoothQuant and Atom, plus the first results on vision-language models. For each method it explains the core idea, the reported accuracy cost, and the memory and speed gains, and it closes with where quantization is used in edge and cloud deployment and what remains open, including quantization-aware training for large models.

CONTENTS

1. Introduction
2. Quantization Overview for LLMs and VLMs
3. Post-Training Weight-Only Quantization Methods
4. Quantization of Weights *and* Activations
5. Quantization for Vision-Language Models
6. Benchmarks and Comparative Performance
7. Real-World Applications and Outlook
8. References

1. Introduction

Large Language Models (LLMs) and Vision-Language Models (VLMs) have achieved remarkable performance on diverse tasks, but their massive size leads to prohibitive memory and computational requirements for inference. For example, GPT-3 with 175 billion parameters requires around 350 GB of memory in half precision, far exceeding the capacity of typical single GPUs [1,2]. Quantization, the practice of reducing the precision of model weights (and sometimes activations), has emerged as a critical technique to compress these models and speed up inference, enabling deployment on resource-constrained environments (edge devices) and cost-efficient serving in the cloud. The key challenge is to reduce model precision (e.g. from 16-bit floats to 8-bit or 4-bit integers) **without significant loss in accuracy**. Recent research (2023 to 2025) has produced **state-of-the-art quantization methods** that can shrink model size by 2 to 4× (or more) while maintaining *near-original* accuracy and even boosting throughput by several times. This report surveys these advances, focusing on how they work, their implementation details, and their empirical performance on LLMs and multimodal vision-language models.

2. Quantization Overview for LLMs and VLMs

Quantization involves converting high-precision values (e.g. 16-bit or 32-bit floats) into lower precision (such as 8-bit or 4-bit integers). This dramatically reduces memory footprint and can leverage low-precision arithmetic units on modern hardware for faster computation. However, naively quantizing all weights and activations often degrades model accuracy due to rounding errors and out-of-range values [3]. Two broad approaches exist:

- **Post-Training Quantization (PTQ):** The model is quantized *after* full training, without additional weight updates. PTQ is fast and convenient, but low-bit PTQ can incur accuracy loss if not done carefully. Recent PTQ methods introduce clever calibration and error compensation to minimize this loss.
- **Quantization-Aware Training (QAT):** The model is trained or fine-tuned with quantization in the loop (simulating quantization errors during training). This usually yields better accuracy retention, but is *extremely costly* for large models (requiring full or partial retraining of an LLM) [4]. As a result, most recent work for LLMs focuses on PTQ, sometimes augmented with light fine-tuning of a few parameters.

Quantization Schemes: Another axis is what is quantized. Many works focus on *weight-only quantization*, keeping activations in higher precision during inference, since **LLM weights dominate memory**. More aggressive schemes quantize both weights and activations to maximize speedups (using integer matrix multiply units). Quantization can be **uniform** (mapping values linearly to a fixed bit grid) or **non-uniform** (allocating bits adaptively to more important ranges

or values). It can also be mixed-precision, assigning different bit-widths to different layers or even to different subsets of weights within a layer, based on their importance. Below, we review leading techniques, grouping them into weight-only methods and weight+activation methods.

3. Post-Training Weight-Only Quantization Methods

Recent PTQ methods for LLMs achieve very low bit-widths for weights (4-bit or even 3-bit per weight) while preserving accuracy. These methods often leverage insight that not all weights are equally important and that **outlier features** (rare large magnitude activations/weights) cause most quantization error. By identifying and preserving or specially handling these outliers, the model can be quantized more aggressively with minimal performance drop.

- **GPTQ (ICLR 2023):** GPTQ introduced a *one-shot* PTQ method using approximate second-order information (Hessian) to minimize the error caused by quantization [5]. It can quantize large transformers (up to 175B parameters) down to *3 to 4 bits per weight with negligible accuracy degradation* [6]. Notably, GPTQ achieved more than **2× compression** gain over prior methods and was the first to demonstrate running a 175B model on a single GPU with near-original accuracy [6]. In practice, GPTQ's 4-bit models preserve perplexity and accuracy very close to the FP16 baselines. It also yields significant speedups due to reduced memory bandwidth: e.g. about **3.25× faster** inference on an NVIDIA A100 GPU (and ~4.5× on an A6000) compared to FP16 execution [7]. GPTQ's efficient algorithm (available as open-source) made it a popular foundation for LLM compression.
- **AWQ (Activation-Aware Weight Quantization, MLSys 2024 Best Paper):** AWQ targets *4-bit weight quantization* optimized for real-world inference, including on-device usage [8]. The key idea is to protect a small fraction of the most important weights by scaling them, instead of using brute-force mixed precision [9,10]. AWQ finds that only about *0.1 to 1% of weight channels* are "salient" (critical to model accuracy); these are identified by analyzing the **activation distribution** (channels that produce larger activations are more important) rather than just weight magnitudes [9,11]. Rather than leaving these in higher precision (which would require inconvenient heterogeneous kernels), AWQ scales up the salient weight channels during quantization so that their relative quantization error is minimized [12,13]. This approach avoids any retraining or backpropagation, relying only on a small calibration set to collect activation stats. The result is a *hardware-friendly* 4-bit model that **consistently outperforms naive round-to-nearest and even GPTQ** in perplexity across model sizes (7B to 70B) [14]. For example, AWQ applied to LLaMA/Llama-2 models yields lower perplexity than GPTQ's 4-bit variants [14]. Crucially, AWQ generalizes better to *out-of-distribution* tasks, it preserves accuracy on domains like coding, math reasoning, and instruction following, where GPTQ can overfit its calibration set [15,16]. It's also the first quantization method shown to work well on **multi-modal LLMs** (vision-language models) without degradation [8] (more on this in a later section). In terms of efficiency, the authors implement an optimized inference engine called **TinyChat** for 4-bit models, with fused kernels and specialized weight packing. AWQ + TinyChat achieves

>3× faster generation than the HuggingFace FP16 baseline on both high-end GPUs and even mobile-class GPUs [17]. Impressively, it enabled deploying a 70B-parameter LLaMA2 model on a **NVIDIA Jetson Orin (64GB) edge device** with reasonable speed [17], a dramatic testament to the power of quantization in bringing LLMs to edge hardware.

- **OWQ (Outlier-Aware Weight Quantization, AAAI 2024):** OWQ explicitly targets the outlier problem by using a *sensitivity-aware mixed-precision* strategy [18]. It identifies a small subset of weights that are highly sensitive to quantization (often those associated with outlier activations) and keeps them in higher precision, while quantizing the bulk of weights to low precision [18]. In effect, OWQ "prioritizes" important weights by not quantizing them as aggressively. This approach significantly reduces quantization error. The authors report that an effective precision of **3.1 bits** (i.e., mostly 3-bit with a few weights in higher precision) can match the performance of a full **4-bit baseline** optimized by earlier methods [19]. In particular, OWQ's 3.1-bit models perform *on par with* 4-bit GPTQ/OPTQ models in accuracy [19], showing the benefit of judiciously not rounding the outliers. Moreover, OWQ introduces a lightweight fine-tuning method called **Weak Column Tuning (WCT)** that can adapt a quantized model to downstream tasks with minimal overhead [20]. WCT fine-tunes a small number of additional parameters (one per column of certain layers) to recover task-specific accuracy lost in quantization, offering a practical way to specialize low-bit LLMs. This combination of mixed precision and tunable columns makes OWQ a flexible solution for both efficient *inference* and *fine-tuning* of LLMs [21].

- **SpQR (Sparse-Quantized Representation, arXiv 2023):** SpQR (by Dettmers et al.) introduced a novel **hybrid format** for near-lossless compression of LLM weights [22]. It works by **isolating outlier weights** that would cause large errors if quantized, and storing those in higher precision, while quantizing all other weights to 3 to 4 bits [23]. The outliers are stored in a *sparse* structure (only 1 to 2% of weights, for example, might be kept in 16-bit) so the overall memory savings remain high. Using this approach, SpQR achieved for the *first time* **near-lossless** compression: <1% perplexity increase compared to the full model, across various LLMs [24]. For instance, compressing LLaMA-65B or Falcon-40B to 3 to 4 bits with SpQR results in under 1% relative perplexity loss [24], effectively maintaining accuracy. This makes it feasible to run a **33B parameter model on a single 24 GB GPU with no performance degradation**, which was previously impractical [24]. SpQR's custom GPU inference kernels also yield faster throughput than FP16 (the paper notes about **15% speedup** over FP16 baseline inference) while providing **>4× memory reduction** [24]. In summary, SpQR demonstrates that by blending sparsity (for problematic weights) with quantization for the rest, one can push to extremely low bit-widths without the downsides.

- **SqueezeLLM (ICML 2024):** SqueezeLLM takes a comprehensive approach, combining *non-uniform quantization* and sparse outlier handling to enable ultra-low precision. It introduces a **Dense-and-Sparse quantization** framework: most weights are quantized to low bit rates, but "sensitive" weights are either assigned higher precision or encoded in a sparse form,

somewhat similar to SpQR. Uniquely, SqueezeLLM uses a **sensitivity-driven bit allocation** based on second-order information (Hessian), effectively choosing how many bits to give each layer or weight group for minimal loss [25]. This allows it to push to an average of ~3-bit precision without accuracy loss. In fact, the authors claim *lossless compression up to 3-bit* per weight for LLaMA models [26]. Compared to prior methods under the same memory constraints, SqueezeLLM's clever bit-allocation reduced the perplexity gap to FP16 by up to *2.1*× (meaning it preserves accuracy significantly better) [27]. In practice, a 3-bit quantized LLaMA by SqueezeLLM outperforms other 3 to 4 bit schemes like GPTQ or AWQ in perplexity. Thanks to the reduced model size, it also improves speed: tested on an NVIDIA A6000, SqueezeLLM's quantized models ran **2.3× faster** than the FP16 version [27]. This method demonstrates the benefit of *non-uniform quantization*: allocating more bits where needed (or using sparse high-precision for outliers) instead of a flat uniform quantization for all weights.

Accuracy and Performance: It's worth noting that all the above weight-only quantization methods prioritize preserving accuracy to within a few percent of the original model. Techniques like AWQ, OWQ, SpQR, and SqueezeLLM all report **perplexity or accuracy drops of well under 1-2%** on standard language benchmarks, even at 3 to 4 bit weights [24,19]. They improve upon earlier methods like naive round-to-nearest quantization (which might suffer large losses at 4-bit) and even upon GPTQ in many cases [14]. A recent summary noted that OWQ, SpQR, and SqueezeLLM achieve *slightly better* accuracy than AWQ, which in turn outperforms GPTQ, making these advanced methods essentially *near-lossless* in compressing LLMs [28]. The trade-off is often computational complexity in the offline quantization process or a more complex model format. In terms of runtime speed and efficiency, weight-only quantization already yields benefits by reducing memory usage (which is the main bottleneck for single-batch inference). For example, GPTQ's 4-bit models more than double inference throughput on GPUs [7], and AWQ's 4-bit engine triples throughput on both high-end and mobile GPUs [17]. However, to fully unlock hardware acceleration (tensor cores, etc.), quantizing activations as well can provide further speedups, as discussed next.

4. Quantization of Weights *and* Activations

While weight-only quantization drastically cuts memory, the activations (intermediate layer outputs) are still typically in FP16/FP32, which limits end-to-end speed gains and memory movement improvements. Quantizing activations to INT8 or INT4 is the next step, but it is challenging because activations in LLMs can have **outlier values** (especially in attention layers) that are not well-handled by naive quantization (clipping or scaling may distort them). Nevertheless, recent advances have made **8-bit activation quantization** practical for LLMs, and even explored 4-bit activations.

- **LLM.int8() (Dettmers et al., 2022):** This was an earlier milestone demonstrating that 8-bit *weight+activation* inference is possible for Transformers by handling activation outliers. The solution was to detect outlier activation channels and dynamically compute those in higher precision, while quantizing the rest in INT8. LLM.int8 showed nearly no accuracy loss on models like OPT-175B when using 8-bit weights and activations [29]. This work proved that transformer layers could be executed in INT8 precision on GPU, leveraging NVIDIA's INT8 Tensor Cores, *as long as* one accounts for the few large activation values.
- **SmoothQuant (ICML 2023):** SmoothQuant generalized the idea of handling outliers by proposing an **offline "smoothing" of activation outliers** via weight transformation [30]. The authors observed that weights are easier to quantize uniformly, whereas activations have problematic outlier channels. SmoothQuant introduces a mathematically equivalent transformation: by *dividing out* the activation outlier magnitude from the weights (scaling certain weight channels up or down accordingly), the magnitude variance is shifted from activations into weights [30]. In effect, it "smooths" the activation distribution, making *both* weights and activations amenable to INT8 quantization. Using this approach, SmoothQuant enabled **8-bit precision for all weights and activations** in models like OPT, BLOOM, LLaMA-2, etc., with *minimal accuracy loss* [31]. This unlocked significant hardware acceleration: e.g., INT8 matrix multiplies on A100 GPUs can utilize fast Tensor Cores. The paper reported up to **1.56× speedup** and **2× memory reduction** for LLM inference with negligible accuracy drop when using SmoothQuant (W8A8) compared to FP16 [31]. Notably, they managed to serve a **530B parameter model on a single machine** with this method, something impossible without such compression [31]. SmoothQuant proved to be a *turn-key* solution, requiring no retraining, thus making INT8 quantization a practical choice for production systems that need faster throughput [31]. (It is worth mentioning that SmoothQuant's focus was on enabling INT8 *for all* layers, whereas LLM.int8() had left a small fraction of outlier computations in higher precision; SmoothQuant made the process uniform and automated via weight scaling.)
- **Low-bit (4-bit) Weight+Activation Quantization:** Going below 8-bit for activations is even harder, but research in 2023-2024 has started exploring it. Some early attempts, such as **OmniQuant** and **QLLM** in 2023, tried to achieve **4-bit weights and activations** on LLMs. However, these methods saw a significant jump in perplexity (accuracy drop) compared to FP16, indicating that 4-bit activations without special handling were too lossy [32]. The challenge is that with such low precision, the quantization errors in activations can compound and destabilize the model's computations.

Atom (MLSys 2024) is a state-of-the-art solution that targets *4-bit weight and activation* quantization while retaining high accuracy. The **Atom** framework uses a combination of techniques: (1) **Mixed precision** for a small subset of the most important activations and weights (similar in spirit to OWQ, it keeps a few values in higher precision rather than uniform 4-bit) [33]; (2) **Fine-grained grouping** of weights and activations for quantization, which allows tailoring the quantization scale to small groups and reducing error [34]; (3) **Dynamic quantization of**

activations at runtime (calculating the optimal scale per input batch on the fly, rather than using a fixed scale from calibration) [35]; and (4) a full-stack inference engine to efficiently support the above (fused operators, quantized *KV cache* to save memory, etc.). With these, Atom manages to quantize LLaMA models to 4-bit weights and activations with *minimal perplexity increase*. In fact, across model sizes from 7B to 65B, **Atom's 4-bit perplexity is nearly as low as the FP16 baseline** (virtually no loss) [36]. This is a remarkable result at 4-bit *full* quantization, which prior methods struggled with [32]. Consequently, Atom can leverage INT4 tensor operations on newer hardware: the authors demonstrate up to **7.7× throughput improvement** for LLM inference with only a slight accuracy loss, bringing low-latency LLM serving closer to reality [33,37]. Atom essentially represents the cutting edge of quantization, showing that even 4-bit activations are feasible when combining mixed-precision and adaptive strategies.

In summary, for cloud-based inference on GPUs with tensor core support, using **8-bit quantization (SmoothQuant, etc.)** is a popular choice to get big speedups without sacrificing accuracy. Many production deployments now use int8 quantization to maximize throughput. The more experimental **4-bit full quantization** methods (Atom, etc.) are pushing the envelope further, promising even greater efficiency, though currently with more complex implementation and slightly higher risk of accuracy loss if not carefully managed. It's expected that as hardware support for lower precision improves (NVIDIA's Hopper architecture already introduced FP8 support, and future cores may support INT4 arithmetic at scale), these techniques will become increasingly practical.

5. Quantization for Vision-Language Models

Vision-Language Models (VLMs), such as CLIP, Flamingo, BLIP-2, PaLM-E, or multi-modal LLMs like GPT-4, combine image and text processing components. Quantizing VLMs involves compressing not only the language model portions (often a transformer decoder or similar) but also the vision backbones (CNN or Vision Transformer encoders) and any cross-modal fusion layers. Historically, quantization of vision models (e.g. CNNs) to INT8 has been well-studied and is standard in many deployments (e.g. TFLite INT8 models for mobile). However, the **combination** of vision and language can create new challenges: for instance, image feature distributions might differ from text, and multimodal transformers (like Flamingo) have even more parameters to compress. Recent quantization research has begun tackling these challenges, showing that the same advanced PTQ techniques can be extended to VLMs.

A notable case is **OpenFlamingo-9B**, a large vision-language model (based on a 9B parameter language model with a vision encoder for image inputs). The AWQ method discussed earlier was tested on OpenFlamingo to verify its efficacy on multi-modal models. Using 4-bit weight quantization, AWQ achieved excellent results on image-captioning tasks: it *outperformed prior quantization methods under zero-shot and few-shot settings* on the COCO Caption benchmark [38]. In other words, a quantized VLM using AWQ was able to generate image captions with accuracy

very close to the full-precision model, even in challenging evaluation scenarios, while other quantization schemes struggled. This demonstrates that **activation-aware weight quantization generalizes to different modalities**, by focusing on the combined model's activation statistics, AWQ preserved the crucial weights for multimodal reasoning [15,38]. The quantized OpenFlamingo saw only minimal drops in BLEU/METEOR scores for captioning, proving that large VLMs can indeed be compressed to 4-bit without losing their cross-modal understanding capabilities [38].

For vision encoders specifically (like a CLIP ResNet or ViT), quantization techniques from the computer vision domain (PTQ with calibration or QAT with augmented training) can be applied. Many vision models are quite robust to 8-bit quantization, and even 4-bit for certain layers, if properly calibrated. The biggest challenge often lies in the language model part, which tends to be more sensitive to quantization (due to the heavy use of self-attention and the aforementioned outlier features). Thus, methods like AWQ, SpQR, etc., originally developed for LLMs, are directly beneficial to VLMs by handling the transformer-based text and multi-modal layers. Meanwhile, the vision backbone can often be quantized with standard methods (as these backbones resemble classification models which have known quantization solutions).

On real-world devices, quantized VLMs promise to bring multimodal AI to edge applications. For example, the TinyChat framework from AWQ is not limited to pure LLMs: it's an **inference engine for 4-bit LLM/VLM** models [39]. The authors demonstrated TinyChat running a 4-bit multimodal model on a Jetson Orin, enabling tasks like image captioning or visual question-answering on a device with limited compute [17]. This suggests that in the near future, we could have phones or AR glasses running compact VLMs locally, thanks to quantization.

6. Benchmarks and Comparative Performance

To put the above methods in perspective, Table 1 summarizes several state-of-the-art quantization techniques for LLMs, highlighting their bit precision and reported performance/accuracy outcomes:

Method (Year)	Precision	Accuracy Impact	Notable Results
GPTQ (Frantar *et al.*, 2023)	3 to 4 bit weights (PTQ)	Negligible loss (perplexity drop nearly zero) [6]	175B GPT-3 compressed to 3 to 4 bits with ~ *no accuracy degradation*; **>3×** speedup vs FP16 (A100) [7]. Enabled single-GPU inference for 175B model [6].

Method (Year)	Precision	Accuracy Impact	Notable Results
AWQ (Lin *et al.*, 2024)	4 bit weights (PTQ)	Negligible loss; outperforms GPTQ on LLMs [14]; excellent on multi-modal [8]	Protects ~1% top weights via scaling (no retraining). **Better perplexity** than GPTQ across 7B to 70B models [14]. First to quantize multi-modal LLMs with minimal loss [8]. >3× faster inference on GPU (with TinyChat) [17]. Runs Llama-2 70B on a 64GB Jetson (edge) [17].
OWQ (Lee *et al.*, 2024)	~3.1 bit avg (mixed)	Essentially lossless: matches 4-bit GPTQ accuracy [19]	Mixed precision: keeps outliers in high-precision. **3.1-bit OWQ = 4-bit GPTQ** in quality [19]. Includes "weak column" fine-tuning to adapt quantized LLMs to tasks [40].
SpQR (Dettmers *et al.*, 2023)	3 to 4 bit weights (PTQ + sparse)	<1% perplexity increase (*near-lossless*) [24]	Stores outlier weights in FP16 sparsely, rest in 3 to 4 bits. Achieved **0% degradation** on 33B model (perplexity loss <1%) [24]. >4× memory reduction, slight speedup (~15%) vs FP16 [24].
SqueezeLLM (Kim *et al.*, 2024)	3 bit avg (PTQ)	No discernible loss (designed for *lossless* compression) [25]	Non-uniform allotment of bits + sparse outliers. **Perplexity gap halved (2.1× better)** vs prior art under same memory budget [27]. Achieved 2.3× speedup on GPU vs FP16 [27].
SmoothQuant (Xiao *et al.*, 2023)	8 bit weights & acts (PTQ)	Negligible loss (<0.5 pp on BLEU, etc.) [31]; slightly lower accuracy than 4-bit weight-only methods on some tasks [28]	**Enabled int8 for all layers** via activation "smoothing." Up to **1.56× faster** and 2× memory savings on GPT/BLOOM models [31]. Allows full INT8 pipeline (uses Tensor Cores); served 530B model on one node [31].
Atom (Ye *et al.*, 2024)	4 bit weights & acts (PTQ)	Minimal loss (perplexity ~FP16 across 7B to 65B models) [36]	Mix of 4-bit and higher precision for outliers + group-wise dynamic quantization. Maintains *near-FP16 accuracy* at 4-bit [36]. Achieved **4 to 8×** throughput improvements with custom kernels [33,37].

Table 1: Comparison of recent quantization methods for large models. PTQ = post-training quantization (no model fine-tuning needed). All methods listed prioritize accuracy retention, generally keeping drops very small (a few percent or less on perplexity or task metrics). Speedups

are as reported on specific hardware and may vary with batch size and model. *(Sources: as cited inline above.)*

As seen in the table, the cutting-edge techniques in 2023 to 2024 have made 4-bit weight quantization **essentially lossless** for most purposes, and even full 8-bit (or 4-bit) quantization including activations is coming within reach. A rough ranking from various reports [28] is that methods like OWQ, SpQR, SqueezeLLM tend to achieve the *absolute highest accuracy* (often at the cost of more complex encoding or slightly slower runtimes due to handling outliers), while AWQ offers a very attractive balance of simplicity, accuracy, and speed. GPTQ, as the pioneer, is slightly behind these newer methods in accuracy on very low bits, but is still highly competitive and widely used. SmoothQuant, while not reaching the same low bit-width, excels in leveraging hardware for speed (int8 compute), making it very practical. In terms of runtime efficiency, using int8 for activations gives SmoothQuant an edge, it was reported as the fastest in one comparison [41], followed by AWQ (with its optimized 4-bit kernels). The more intricate schemes (OWQ, SpQR, etc.) involve extra steps (like checking a table of outliers or switching precision) that can make them slightly slower, but they shine in scenarios where every bit of accuracy matters.

7. Real-World Applications and Outlook

Quantization has quickly transitioned from a research topic to a **real-world enabler** for deploying large models. On the *edge,* quantization is often the only viable way to run an LLM or VLM. For instance, getting a 7B to 70B model to run on a consumer GPU or mobile SoC requires 4-bit weights (or lower) given limited RAM. Techniques like AWQ and SpQR have made this possible without sacrificing user experience, bringing personalization and privacy (via on-device AI) closer to reality. We have already seen 13B+ parameter chatbots running on smartphones in demonstrations, using 4-bit quantization and clever runtime optimization.

In *cloud inference*, quantization directly translates to cost savings: a quantized model uses less memory (fitting more models or instances per GPU) and can utilize faster integer math units. Many inference frameworks now integrate support for GPTQ or SmoothQuant models, and hardware vendors are introducing optimized int8/int4 libraries. For example, NVIDIA's TensorRT and Intel's Neural Compressor provide recipes for LLM quantization to INT8. As **hardware evolves**, we expect even better support: FP8 and INT4 acceleration will become standard, making 4-bit quantization even more effective. Google's TPUs already support bfloat16 and INT8, and future TPUs/GPUs may specialize in lower precision.

One ongoing area of research is **Quantization-Aware Training for LLMs**, if some amount of fine-tuning is acceptable, it could further close the gap between quantized and full-precision models. Early work (e.g. Liu et al. 2023 on LLM-QAT) indicates that a brief fine-tuning of a quantized 8-bit model can recover virtually all lost accuracy [42]. However, QAT for 4-bit models remains challenging due to instabilities in training. Another frontier is **extreme quantization**: 2-bit or 3-

bit models. GPTQ showed even 2-bit weights can be attempted with careful error compensation [43], but usually at a larger accuracy cost. Some specialized models or smaller networks might tolerate such ultra-low precision, but generally, 4-bit appears to be the sweet spot for now between size and accuracy.

In conclusion, the 2023 to 2025 period has seen *quantization break through as a key technology* for large models. By retaining accuracy to within a hair's breadth of the original, these methods unlock huge efficiency gains. The fact that a 70B parameter model can run on a handheld device, or that a 500B model can be served on a single server, would have been unthinkable a few years ago, and is now reality thanks to these quantization techniques. As models continue to grow and deployments scale up, expect quantization (along with complementary methods like sparsity and distillation) to play an ever-growing role in making AI accessible and affordable across both cloud and edge environments.

Sources: The report is based on the latest research papers and results in model quantization, including *GPTQ* [5,7], *SmoothQuant* [31], *AWQ* [8], *OWQ* [19], *SpQR* [24], *SqueezeLLM* [27], *Atom* [36], and others as cited above. These advances collectively illustrate the state-of-the-art in quantizing LLMs and VLMs with minimal loss of accuracy while greatly improving efficiency.

References

Numbers in square brackets in the text point to the sources below. The citation numbers follow the original draft, so several numbers can point to the same source.

1. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration (arXiv 2306.00978, ar5iv HTML version)
   `https://ar5iv.org/html/2306.00978v5`
   Cited as [1], [2], [4], [9], [10], [11], [13], [14], [15], [16], [38]

2. LLM Quantization: GPTQ, QAT, AWQ, GGUF, GGML, PTQ (Siddharth Vij, Medium)
   `https://medium.com/@siddharth.vij10/llm-quantization-gptq-qat-awq-gguf-ggml-ptq-2e172cd1b3b5`
   Cited as [3], [12]

3. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers (arXiv 2210.17323)
   `https://arxiv.org/abs/2210.17323`
   Cited as [5], [6], [7], [43]

4. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration (arXiv 2306.00978)
   `https://arxiv.org/abs/2306.00978`
   Cited as [8], [17], [39]

5. OWQ: Outlier-Aware Weight Quantization for Efficient Fine-Tuning and Inference of Large Language Models (arXiv 2306.02272)
   https://arxiv.org/abs/2306.02272
   Cited as [18], [19], [20], [21], [40]

6. SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression (arXiv 2306.03078)
   https://arxiv.org/abs/2306.03078
   Cited as [22], [23], [24]

7. SqueezeLLM: Dense-and-Sparse Quantization (arXiv 2306.07629)
   https://arxiv.org/abs/2306.07629
   Cited as [25], [26], [27]

8. Which Quantization to Use to Reduce the Size of LLMs? (FriendliAI blog)
   https://friendli.ai/blog/quantization-reduce-llm-size
   Cited as [28], [41]

9. Atom: Low-Bit Quantization for Efficient and Accurate LLM Serving (MLSys 2024)
   https://proceedings.mlsys.org/paper_files/paper/2024/file/5edb57c05c81d04beb716ef1d542fe9e-Paper-Conference.pdf
   Cited as [29], [32], [33], [34], [35], [36], [37], [42]

10. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models (arXiv 2211.10438)
    https://arxiv.org/abs/2211.10438
    Cited as [30], [31]